

2. Easy Maintenance and Creation of REST Endpoints: With annotations like `@RestController`, `@GetMapping` and `@PostMapping`, creating REST endpoints is straightforward.

Example:

```
@RestController
@RequestMapping("/api")
public class MyController {
    @GetMapping("/hello")
    public String sayHello() {
        return "Hello, World!";
    }
}
```

```
package com.example.demo.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class DemoController {

    @GetMapping("/hello")
    public String helloWorld() {
        return "Hello World!";
    }
}
```

We have used the below annotations in our controller layer. Here in this example, the URI path is */hello*.

- **@RestController:** Combines `@Controller` and `@ResponseBody`, used to create RESTful web services.
- **@RequestMapping / @GetMapping:** Maps HTTP requests to handler methods.
- **Return Type:** Returned data is written directly to the HTTP response body (usually JSON or plain text).

Simple example about request param:

```
package com.jloka.jloka;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;

@SpringBootApplication
@RestController
public class JlokaApplication {

    public static void main(String[] args) {
        SpringApplication.run(JlokaApplication.class, args);
    }

    @GetMapping("/hello")
    public String getMethodName(@RequestParam(name="name", defaultValue =
"World") String name) {
        return String.format("Hello %s!", name);
    }
}

*****
```

Jackson is included by default by the web starter.

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>

*****
```

The `Greeting` object must be converted to JSON. Thanks to Spring's HTTP message converter support, you need not do this conversion manually. Because `Jackson` is on the classpath, Spring's `JacksonJsonHttpMessageConverter` is automatically chosen to convert the `Greeting` instance to JSON.

Greeting.java:

```
package com.jloka.jloka.resources;

public record Greeting(long id, String content) { }

-----

package com.jloka.jloka.controllers;

import java.util.concurrent.atomic.AtomicLong;
import org.springframework.web.bind.annotation.RestController;
import com.jloka.jloka.resources.Greeting;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;

@RestController
public class GreetingController {
    private static final String template = "Hello, %s!";
    private final AtomicLong counter = new AtomicLong();

    @GetMapping("/greeting")
    public Greeting greeting(@RequestParam(defaultValue = "World") String name) {
        return new Greeting(counter.incrementAndGet(), template.formatted(name));
    }
}
```

`@SpringBootApplication` is a convenience annotation that adds all of the following:

- `@Configuration` : Tags the class as a source of bean definitions for the application context.
- `@EnableAutoConfiguration` : Tells Spring Boot to start adding beans based on classpath settings, other beans, and various property settings. For example, if `spring-webmvc` is on the classpath, this annotation flags the application as a web application and activates key behaviors, such as setting up a `DispatcherServlet` .
- `@ComponentScan` : Tells Spring to look for other components, configurations, and services in the `com/example` package, letting it find the controllers.

The `main()` method uses Spring Boot's `SpringApplication.run()` method to launch an application. Did you notice that there was not a single line of XML? There is no `web.xml` file, either. This web application is 100% pure Java and you did not have to deal with configuring any plumbing or infrastructure.

To implement the jpa example:

For `application.yaml` file:

```
spring:
  application:
    name: JLoka Application
  datasource:
    url:
jdbc:mariadb://localhost:3306/jloka_test_01?createDatabaseIfNotExist=true
    username: root
    password: 123
    driver-class-name: org.mariadb.jdbc.Driver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
    properties:
      hibernate:
        dialect: org.hibernate.dialect.MariaDBDialect
```

For [pom.xml](#) file:

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>org.mariadb.jdbc</groupId>
    <artifactId>mariadb-java-client</artifactId>
    <scope>runtime</scope>
  </dependency>

  <dependency>
    <groupId>org.flywaydb</groupId>
    <artifactId>flyway-core</artifactId>
  </dependency>

  <dependency>
    <groupId>org.flywaydb</groupId>
    <artifactId>flyway-mysql</artifactId>
  </dependency>
</dependencies>
```

For models we must consider the naming styles for jpa, and we must set default constructor:

```
package com.jloka.jloka.models;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity(name = "customers")
public class Customer {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;

    @Column(name= "first_name", nullable = false)
    private String firstName;

    @Column(name="last_name", nullable=false)
    private String lastName;

    protected Customer() {}

    public Customer(String firstName, String lastName) {
        this.firstName = firstName;
        this.lastName = lastName;
    }

    @Override
    public String toString() {
        return String.format(
            "Customer[id=%d, firstName='%s', lastName='%s']",
            id, firstName, lastName);
    }

    public Long getId() {
        return id;
    }

    public String getFirstName() {
        return firstName;
    }

    public String getLastName() {
```

```

        return lastName;
    }
}

```

And for sql file:

```

CREATE TABLE customers (
    id INT AUTO_INCREMENT PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL
);

```

And for repository of customer:

```

package com.jloka.jloka.repositories;

import java.util.List;
import org.springframework.data.repository.CrudRepository;
import com.jloka.jloka.models.Customer;
public interface CustomerRepository extends CrudRepository<Customer, Long> {
    List<Customer> findByLastName(String lastName);
    Customer findById(long id);
}

```

And for Operations we can set the beans:

```

package com.jloka.jloka;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;

import com.jloka.jloka.models.Customer;
import com.jloka.jloka.repositories.CustomerRepository;

@SpringBootApplication
public class JlokaApplication {

    private static final Logger logger =
LoggerFactory.getLogger(JlokaApplication.class);

```

```

public static void main(String[] args) {
    SpringApplication.run(JlokaApplication.class, args);
}

@Bean
public CommandLineRunner demo(CustomerRepository customerRepository) {
    return (args) -> {
        // save customers
        // customerRepository.save(new Customer("JLoka-01", "JLoka-01"));
        // customerRepository.save(new Customer("JLoka-01", "JLoka-01"));
        // customerRepository.save(new Customer("JLoka-01", "JLoka-01"));
        // customerRepository.save(new Customer("JLoka-01", "JLoka-02"));
        // customerRepository.save(new Customer("JLoka-01", "JLoka-03"));

        // fetch all customers
        logger.info("=====");
        logger.info("Getting all customer data");
        customerRepository.findAll().forEach(customer -> {
            logger.info(customer.toString());
        });

        logger.info("=====");
        logger.info("Getting customer by id");
        Customer customer = customerRepository.findById(1L);
        logger.info(customer.toString());

        logger.info("=====");
        logger.info("Getting customers by last name: JLoka-01");
        customerRepository.findByLastName("JLoka-01").forEach(customer02 -> {
            logger.info(customer02.toString());
        });
    };
}
}

*****

```


To Add Controller with jpa:

```
package com.jloka.jloka.controllers;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.ResponseBody;

import com.jloka.jloka.models.Customer;
import com.jloka.jloka.repositories.CustomerRepository;

@Controller
@RequestMapping(path="/demo")
public class CustomerController {

    @Autowired
    private CustomerRepository customerRepository;

    @PostMapping("/add")
    public @ResponseBody Customer addNewCustomer(@RequestParam String firstName,
    @RequestParam String lastName) {
        Customer customer = new Customer(firstName, lastName);
        customerRepository.save(customer);
        return customer;
    }

    @GetMapping("/all")
    public @ResponseBody Iterable<Customer> getAllCustomers() {
        return customerRepository.findAll();
    }
}
```

And the repository:

```
package com.jloka.jloka.repositories;

import java.util.List;
import org.springframework.data.repository.CrudRepository;
import com.jloka.jloka.models.Customer;
public interface CustomerRepository extends CrudRepository<Customer, Long> {
    List<Customer> findByLastName(String lastName);
    Customer findById(long id);
}
```

And the model:

```
package com.jloka.jloka.models;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity(name = "customers")
public class Customer {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;
    @Column(name= "first_name", nullable = false)
    private String firstName;
    @Column(name="last_name", nullable=false)
    private String lastName;

    protected Customer() {}

    public Customer(String firstName, String lastName) {
        this.firstName = firstName;
        this.lastName = lastName;
    }

    @Override
    public String toString() {
        return String.format(
            "Customer[id=%d, firstName='%s', lastName='%s']",
            id, firstName, lastName);
    }
}
```

```

    public Long getId() {
        return id;
    }
    public String getFirstName() {
        return firstName;
    }
    public String getLastName() {
        return lastName;
    }
}

```

And the sql of customer is:

```

CREATE TABLE customers (
    id INT AUTO_INCREMENT PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL
);

```

And if we want to populate the table with data from main springboot app:

```

package com.jloka.jloka;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;

import com.jloka.jloka.models.Customer;
import com.jloka.jloka.repositories.CustomerRepository;

@SpringBootApplication
public class JlokaApplication {
    private static final Logger logger =
        LoggerFactory.getLogger(JlokaApplication.class);

    public static void main(String[] args) {
        SpringApplication.run(JlokaApplication.class, args);
    }

    @Bean
    public CommandLineRunner demo(CustomerRepository customerRepository) {
        return (args) -> {

```

```

// save customers
// customerRepository.save(new Customer("JLoka-01", "JLoka-01"));
// customerRepository.save(new Customer("JLoka-01", "JLoka-01"));
// customerRepository.save(new Customer("JLoka-01", "JLoka-01"));
// customerRepository.save(new Customer("JLoka-01", "JLoka-02"));
// customerRepository.save(new Customer("JLoka-01", "JLoka-03"));

// fetch all customers
logger.info("=====");
logger.info("Getting all customer data");
customerRepository.findAll().forEach(customer -> {
    logger.info(customer.toString());
});

logger.info("=====");
logger.info("Getting customer by id");
Customer customer = customerRepository.findById(1L);
logger.info(customer.toString());

logger.info("=====");
logger.info("Getting customers by last name: JLoka-01");
customerRepository.findByLastName("JLoka-01").forEach(customer02 -> {
    logger.info(customer02.toString());
});
    };
}
}

```